Bài 1: Một bước RNN.

$$W_{xh} x_t + W_{hh} h_{t-1} + b_h$$

$$W_{xh} x_t = \begin{bmatrix} 0,5 \\ 0,1 \end{bmatrix}$$

$$W_{hh} h_{t-1} = \begin{bmatrix} (0,3)0,5 + 0,1.0,2 \\ 0,2(0,5) + 0,5 \cdot 0,2 \end{bmatrix} \cancel{= 0,2}$$

$$= \begin{bmatrix} 0,17 \\ 0,2 \end{bmatrix}$$

$$h_t = \tanh(a_t) = \begin{bmatrix} \tanh(0,67) \\ \tanh(0,3) \end{bmatrix}$$

$$= \begin{bmatrix} 0,585 \\ 0,291 \end{bmatrix}$$

Bài 2. Information flow.

Với câu I did not like this movie, RNN đọc lần lượt từng ~~token~~ token. Trạng thái sau "not" có thể mang thông tin phủ định, khi đọc "like" mô hình có thể kết hợp động từ tích cực với phủ định trước đó, rồi truyền kết quả cho các token tiếp theo

→ không đảm bảo mô hình học được ~~nhiều~~ ~~câu~~ cảnh phủ định trong mọi câu.

Average + word2vec lấy trung bình các ~~từ~~ vector từ. Nó có thể chứa vector của "not" nhưng phép trung bình không giữ được vị trí hay phạm vi ~~đảng~~ tác động của từ phủ định. Do đó khó phân biệt các câu có cùng từ nhưng khác thứ tự và quan hệ cú pháp khác nhau. Hidden state của RNN phụ thuộc vào trạng thái trước đó, nên có thể giữ giữ thông tin về thứ tự.

Bài 3 - Sequence order

"dog bites man" và "man bites dog" có cùng bộ từ. Average Word2Vec cho cùng một vector trung bình vì phép cộng không phụ thuộc vào thứ tự. Với RNN, các bước cập nhật hidden state khác nhau; từ đầu câu khác nhau làm thay đổi trạng thái trước khi đọc đến "bites", nên biểu diễn cuối có thể khác. Tuy nhiên việc phân biệt đúng ai cắn ai còn phụ thuộc vào dữ liệu huấn luyện và những gì mô hình học được.

LUYỆN NÉT CHỮ rèn nết người

Vanishing gradient.

- Khi học từ một token ở xa, gradient phải phải đi ngược qua nhiều bước thời gian. Mỗi bước nhân thêm Jacobian của phép cập nhật hiddent state, với tanh, đạo hàm có độ lớn tối đa 1 và thường nhỏ hơn 1 -> Tích hệ số đó nhỏ dần về 0.

-> Khiến RNN khó gán ảnh hưởng của token đầu câu cho dự án cuối câu. Nếu các hệ số quá lớn, gradient cũng có thể bùng nổ

LSTM. - LSTM có đường truyền cell state cùng các gate để giữ hoặc bỏ thông tin qua nhiều bước, nên thường xử lý phụ thuộc xa dài tốt hơn simple RNN. Gate chỉ giảm khó khăn này, không đảm bảo mọi phụ thuộc dài đều được nhớ.

Bài tập tính toán - LSTM gates

$$c_t = 0{,}9 \cdot 0{,}8 + 0{,}2 \cdot 0{,}5 = 0{,}82$$

$$c_t = 0{,}2$$

- Nếu forget gate gần 1, model có xu hướng

HONGHA

giữ lại phần lớn thông tin từ cell state cũ.

- Nếu forget gate gần 0, model có xu hướng loại bỏ phần lớn thông tin.